Tabular Data Understanding

A comprehensive report submitted in partial fulfillment
of the requirements for the degree of

Doctor of Philosophy
in
Computer Science

by

Anupreksha Jain
2025801010
anupreksha.j@research.iiit.ac.in

Advisor: Dr. Manish Shrivastava



Language Technologies Research Center
International Institute of Information Technology Hyderabad
500 032, India

2026

Copyright © Anupreksha Jain, 2026
All Rights Reserved

Abstract

Tables are among the most pervasive formats for structured information, underpinning enterprise analytics, scientific reporting, and everyday data exchange. Yet reasoning over them with large language models (LLMs) is fundamentally difficult: models consume one-dimensional token sequences, whereas a table's meaning is carried jointly by its rows, columns, headers, and the relationships among cells. The difficulty is compounded by scale. Current methods are demonstrated on small, clean tables (often only a handful of columns and a few dozen rows), whereas the data that matters in practice lives in enterprise databases whose schemas are large, cryptic, and obsolescent and whose tables are far too big to fit in a model's context. Closing this gap between demonstration and deployment, making tabular understanding work at *industry scale*, is the motivation of this work.

This report surveys how the field bridges the representation mismatch and organizes it around two complementary pillars that together address that gap: *schema understanding*, the mapping of natural-language intent onto a table's structural blueprint, and *data understanding*, the problem of reasoning faithfully over table content, whose canonical task is Table Question Answering. Because how a table is represented to a model (serialization, data schema, image, a specialized table encoder, or a graph) bears most directly on reasoning over content, we treat representation within the data-understanding pillar, singling out graph representations as especially promising for making table structure explicit and addressable yet so far demonstrated only on small or document-centric data, whose extension to large tables we take as a motivating direction. We synthesize the literature under the two pillars, tracing a consistent shift from single-pass methods toward multi-step pipelines that decompose, retrieve, verify, and correct, and arguing that the binding constraints are increasingly architectural rather than a matter of raw model capacity.

Against this backdrop we present our contribution to the schema-understanding pillar, Reflect-SQL [1], a self-reflective Text-to-SQL system that constructs a semantic knowledge base over the schema, retrieves relevant elements iteratively, and synthesizes queries under syntactic and multi-dimensional semantic validation, scoring correctness with an LLM-as-a-judge and without a gold reference. By grounding reflection in the checkable signals the SQL setting affords, Reflect-SQL reaches 72.03% execution accuracy on the industry-scale BIRD benchmark.

Finally, we lay out an extensive research agenda across both pillars: for schema understanding, continually evolving knowledge bases, retrieval that scales by addressing pre-computed structure rather than searching at query time, cross-database join reasoning, and reliable yet inexpensive reflection; and for data understanding, our next direction, representing tabular data at scale as a graph so that evidence

retrieval becomes addressing rather than search, with robustness on complex table structures and higher-order reasoning as the targets and a large-scale Table Question Answering benchmark as the instrument to measure them. The unifying thesis is that progress in tabular data understanding depends less on ever-larger models than on the architecture that supplies them with the right structure and the right evidence.

Contents

Chapter Page

1 Introduction . . . 1
1.1 Objective and scope . . . 3
1.2 Contributions . . . 3
1.3 Structure of the report . . . 4

2 Literature Survey . . . 5
2.1 Schema Understanding . . . 5
2.1.1 Knowledge-base and metadata construction . . . 6
2.1.2 Schema linking and retrieval . . . 6
2.1.3 From encoder–decoder models to LLM pipelines . . . 6
2.1.4 Self-reflection for structured generation . . . 7
2.1.5 From benchmarks to the enterprise . . . 7
2.2 Data Understanding . . . 7
2.2.1 Representing table content . . . 8
2.2.2 Graph representations . . . 8
2.2.3 Table Question Answering . . . 9
2.2.4 Adjacent tasks . . . 9
2.2.5 Open challenges . . . 9
2.2.6 From benchmarks to the enterprise . . . 10

3 Current Progress: Reflect-SQL . . . 11
3.1 Framework Overview . . . 12
3.2 Knowledge Base Construction and Refinement . . . 12
3.2.1 Construction . . . 13
3.2.2 Iterative refinement . . . 13
3.3 Iterative RAG Retrieval . . . 14
3.3.1 Hierarchical table and column retrieval . . . 14
3.3.2 Relevance and completeness scoring . . . 15
3.3.3 Query refinement . . . 15
3.4 SQL Query Generation . . . 15
3.4.1 Syntactic validation and correction . . . 16
3.4.2 Semantic validation . . . 16
3.5 Entailment . . . 16
3.5.1 Entailment scoring . . . 16
3.5.2 Entailment decision . . . 16

3.6 Mitigating Bias in LLM-based Evaluation . . . 16
3.7 Model Parameter Optimization . . . 17
3.8 Evaluation . . . 17
3.8.1 Experimental setup . . . 17
3.8.2 Results . . . 17
3.8.3 Human evaluation . . . 18
3.8.4 Comparison with state of the art . . . 18
3.9 Ablation Study . . . 19
3.10 Summary . . . 19

4 Future Directions . . . 20
4.1 Future Directions in Schema Understanding . . . 20
4.1.1 From static to continually evolving knowledge bases . . . 20
4.1.2 Retrieval at industry scale . . . 21
4.1.3 Cross-database and multi-table join reasoning . . . 21
4.1.4 Reliable, reference-free verification . . . 21
4.1.5 The cost of reflection . . . 22
4.1.6 Beyond SQL . . . 22
4.2 Setting a Direction for Data Understanding . . . 22
4.2.1 Representing large-scale tabular data as a graph . . . 23
4.2.2 Evidence retrieval as addressing over the graph . . . 23
4.2.3 Robustness on complex table structures . . . 24
4.2.4 Reasoning beyond retrieval and arithmetic . . . 24
4.2.5 Generalization across representations . . . 24
4.2.6 A large-scale benchmark for Table Question Answering . . . 24
4.3 Summary . . . 25

5 Conclusion . . . 26

Bibliography . . . 28

List of Figures

Figure Page

3.1 High-level architecture of the Reflect-SQL framework. 12
3.2 Example JSON specification of the knowledge base. 13
3.3 Iterative retrieval loop. 14
3.4 Iterative SQL generation loop. 15

List of Tables

Table Page

3.1 Example of knowledge base refinement for BIRD [8]. 14
3.2 Optimized parameters using grid search. 17
3.3 Accuracy (%) of various models on the BIRD dataset. Top performers in each category are highlighted. 18
3.4 Example of SQL refinement driven by entailment feedback. 18
3.5 Comparison with state-of-the-art methods on the BIRD dev set (execution accuracy %). 19
3.6 Ablation results showing the contribution of each module. 19

List of Selected Papers

The following are the key papers that this report draws upon and that form the basis for discussion, organized by the two pillars around which the report is structured.

Foundations and Representations

[S1] X. Wu, A. Ritter, and W. Xu, **"Tabular Data Understanding with LLMs: A Survey of Recent Advances and Challenges"**, *arXiv:2508.00217*, 2025.

[S2] Y. Sui, M. Zhou, M. Zhou, S. Han, and D. Zhang, **"Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study"**, in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM)*, 2024.

Schema Understanding (Text-to-SQL)

[S3] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, et al., **"Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task"**, in *Proceedings of EMNLP*, 2018.

[S4] J. Li, B. Hui, G. Qu, J. Yang, B. Li, et al., **"Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs (BIRD)"**, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[S5] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, et al., **"CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL"**, *arXiv:2410.01943*, 2024.

[S6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, **"Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection"**, *arXiv:2310.11511*, 2023.

Data Understanding

[S7] P. Pasupat and P. Liang, **"Compositional Semantic Parsing on Semi-Structured Tables"**, in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2015.

[S8] Z. Cheng, H. Dong, Z. Wang, R. Jia, J. Guo, et al., **"HiTab: A Hierarchical Table Dataset for Question Answering and Natural Language Generation"**, in *Proceedings of ACL*, 2022.

[S9] W. Chen, H. Wang, J. Chen, Y. Zhang, H. Wang, et al., **"TabFact: A Large-Scale Dataset for Table-Based Fact Verification"**, in *International Conference on Learning Representations (ICLR)*, 2020.

Graph Representation of Tabular Data

[S10] Q. Li, C. Huang, S. Li, Y. Xiang, D. Xiong, and W. Lei, **"GraphOTTER: Evolving LLM-Based Graph Reasoning for Complex Table Question Answering"**, in *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, 2025.

[S11] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi, **"G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering"**, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[S12] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, et al., **"From Local to Global: A Graph RAG Approach to Query-Focused Summarization"**, *arXiv:2404.16130*, 2024.

Chapter 1

Introduction

Tables are among the most pervasive means of organizing information. From relational databases that power enterprise systems, to spreadsheets used in finance and science, to the info-boxes and result tables embedded in web pages and research papers, a large fraction of the world's structured knowledge is stored in tabular form. Enabling machines to *understand* this data (to answer questions over it, verify claims against it, summarize it, and translate natural-language intent into precise queries against it) is therefore a foundational problem for natural language processing and a prerequisite for democratizing access to data. The stakes are highest in the enterprise: organizations sit on vast operational databases and warehouses, yet the ability to interrogate them is gated behind specialists who can write SQL and who know what each cryptic column actually means. Letting any analyst or decision-maker ask a question in natural language and get a trustworthy answer over data at this scale is the practical prize that motivates the field, and the gap between that promise and what current systems deliver is the problem this report sets out to frame.

The recent progress of Large Language Models (LLMs) and Multimodal Large Language Models (MLLMs) has renewed interest in tabular data, but tables expose a fundamental mismatch with how these models process input. LLMs are trained on *linear*, one-dimensional token sequences, whereas a table is inherently *two-dimensional*: its meaning is carried jointly by rows, columns, headers, and the relationships between cells [2]. Compounding this, tables are structurally *flexible*. They range from well-normalized relational database tables, to hierarchical and multi-layered spreadsheets, to semi-structured info-boxes linked to free text. This diversity of format and purpose has driven the field toward a fragmented landscape of specialized representations, tasks, and datasets rather than universal solutions, making the area difficult to navigate and models difficult to generalize [2].

Before any reasoning can occur, a two-dimensional table must be converted into a form a model can consume. The choice of this input representation is not neutral: recent studies show that model performance can swing substantially depending on how closely the chosen format aligns with the data seen during pre-training [3]. The literature broadly distinguishes four families of representation: *serialization* (e.g., HTML, JSON, Markdown, or LaTeX renderings of the full table), *data schema* (the structural blueprint of the data, such as SQL `CREATE` statements or pandas dataframes), *image*-based formats, and specialized *table encoders* [2]. These representations trade off completeness against compactness:

serialization and images preserve content but are bounded by context length and resolution, whereas a schema-based view bypasses length limits by exposing only structure, at the cost of the concrete values. A limitation that is easy to overlook is one of scale: these representations, and the methods built on them, are overwhelmingly validated on *small* tables, often no larger than four or five columns and a few dozen rows. Real-world tables are nothing like this. Enterprise databases hold hundreds of columns and thousands to millions of rows, at which point serialization overflows the context window, image renderings blur past legibility, and even schema-only views strain as the number of tables and relationships explodes. A further, more structural option casts the table as a *graph*, making its layout explicit and addressable rather than implicit in a flattened token stream; this is a promising foundation, but current graph methods are demonstrated only on small or document-centric data, and extending them to large tables is part of the motivation this report develops. The chasm between the toy tables on which solutions are demonstrated and the industry-scale tables on which they must actually work is the central motivation of this report.

This report studies tabular data understanding through two complementary lenses that together span this space:

- **Schema understanding** is concerned with the *structural blueprint* of tabular data (table and column names, data types, primary and foreign keys, and inter-table relationships) and with mapping a user's natural-language intent onto that structure. The difficulty is defined by industry reality: enterprise schemas are large (thousands of tables and tens of thousands of columns), their naming is cryptic and undocumented (e.g., columns named `A2` or `dname`), and they are *obsolescent*, drifting as columns are added, repurposed, or abandoned while documentation lags behind. The central challenges are therefore disambiguating obscure and stale schemas, retrieving the relevant schema elements for a query from among tens of thousands of candidates, and generating structurally and semantically valid operations such as SQL over them. Text-to-SQL is the canonical task in this pillar.

- **Data understanding** is concerned with reasoning over the *content* of tables (the values, their aggregation, and the trends, outliers, and relationships they encode) often in combination with surrounding text. Here the challenges are higher-order reasoning that goes beyond retrieval and arithmetic, and robustness to complex inputs such as hierarchical, long, and multi-table settings. The same scale problem recurs: methods excel on the small, clean tables of academic benchmarks but degrade sharply once a table is large enough that the handful of answer-bearing cells is buried among thousands of irrelevant ones. Table Question Answering is the canonical task in this pillar, with table fact verification and table-to-text generation as adjacent tasks.

These two pillars are complementary rather than disjoint. A schema is a lossy abstraction of the underlying data, while the data is only interpretable in light of its schema; robust tabular understanding requires both. The current state of the field, however, is uneven, and in a way that sharpens the motivation above. On the schema-understanding side, Text-to-SQL has advanced rapidly from early encoder–

decoder architectures [4] to LLM-based in-context and multi-agent pipelines [5–7], with strong results on benchmarks such as BIRD [8], yet these gains are reported on curated databases and do not transfer cleanly to the obscure, obsolescent, industry-scale schemas described above. On the data-understanding side, several benchmarks are approaching saturation on retrieval-oriented questions over small tables, yet models remain markedly weaker than humans once inputs become structurally complex: on hierarchical and multi-table benchmarks such as HiTab [9] and MULTIHIERTT [10], reported accuracies fall well below human performance. The unifying motivation of this report is therefore to close the gap between demonstration and deployment: to make *both* schema understanding and data understanding work at industry scale, where schemas are large and stale and tables are far too big to fit in a model's context. Schema understanding is the natural first front, and data understanding the next frontier once it is well founded.

1.1 Objective and scope

The objective of this report is to survey and consolidate current research on tabular data understanding with large language models, organized around the two pillars of schema understanding and data understanding, and to situate our own research within this landscape. The report is scoped as follows:

- We treat the taxonomy of tabular input representations (serialization, data schema, image, table encoders, and graph-based representations) within the data-understanding pillar, since this choice bears most directly on reasoning over content, and examine the sensitivity of model behaviour to it.
- Under **schema understanding**, we review knowledge-base and metadata construction for obscure schemas, schema linking and retrieval, and the evolution of Text-to-SQL methods, culminating in retrieval-augmented and self-reflective LLM pipelines.
- Under **data understanding**, we center on Table Question Answering, noting table fact verification and table-to-text as adjacent tasks, and review the open challenges of reasoning beyond retrieval, robustness to complex inputs, and generalization across tabular representations.
- As current progress, we present *Reflect-SQL*, a self-reflection-based framework for Text-to-SQL that operationalizes the schema-understanding pillar, and we use it to motivate data understanding as the forward research direction.
- The report focuses on methods, tasks, and benchmarks at the level needed to frame a research programme; it is not intended as an exhaustive technical account of every approach.

1.2 Contributions

The following are the contributions of this report:

- A structured survey of tabular data understanding that unifies the fragmented literature under two complementary pillars, schema understanding and data understanding, treating the question of how tables are represented to models (including graph-based representations) within the data-understanding pillar.
- A synthesis of the Text-to-SQL and schema-understanding literature, tracing the progression from encoder–decoder models to knowledge-base-driven, retrieval-augmented, and self-reflective LLM frameworks, and identifying where reliability gaps remain.
- A synthesis of the data-understanding literature that consolidates the principal reasoning tasks and articulates three open challenges: limited reasoning beyond retrieval, lack of robustness on complex inputs, and limited cross-representation generalization.
- *Reflect-SQL* [1], our framework for enterprise Text-to-SQL, which introduces a dynamically enriched knowledge base, a feedback-driven retrieval loop, an iterative SQL synthesis loop, and a reference-free entailment check, and which achieves an execution accuracy of 72.03% on the BIRD benchmark.
- An extensive research agenda spanning both pillars: for schema understanding, continually evolving knowledge bases, retrieval that scales by addressing pre-computed structure rather than searching at query time, cross-database join reasoning, and reliable yet inexpensive reflection; and for data understanding, representing tabular data at scale as a graph as the lead direction, with evidence retrieval recast as addressing over that structure and a large-scale Table Question Answering benchmark to measure progress.

1.3 Structure of the report

The remainder of this report is organized into four chapters. Chapter 2 presents the literature survey: organized around the two pillars, it reviews the schema-understanding pillar (knowledge-base construction, schema linking and retrieval, and the evolution of Text-to-SQL from encoder–decoder models to self-reflective LLM pipelines) and the data-understanding pillar (how table content is represented, including graph-based representations, Table Question Answering, and its open challenges), establishing the research context for both pillars in a balanced manner. Chapter 3 reports current progress: it details *Reflect-SQL*, our self-reflection framework for Text-to-SQL, including its knowledge base, retrieval and generation loops, reference-free entailment check, and experimental results on the BIRD benchmark. Chapter 4 sets out an extensive research agenda: it consolidates what schema understanding has achieved and the problems that remain, then charts our next direction in data understanding, centered on representing tabular data at scale (concretely as a graph) and the large-scale benchmark needed to measure it. Chapter 5 concludes by summarizing the findings and restating the report's central thesis: that progress in tabular data understanding depends less on ever-larger models than on the architecture that supplies them with the right structure and the right evidence.

Chapter 2

Literature Survey

This chapter surveys the research landscape of tabular data understanding, organized around the two pillars that structure this report: *schema understanding*, which concerns the structural blueprint of tabular data and the mapping of natural-language intent onto it (Section 2.1); and *data understanding*, which concerns reasoning over the content of tables, canonically instantiated as Table Question Answering (Section 2.2). The two pillars share one underlying difficulty: large language models consume one-dimensional token sequences, whereas a table's meaning is carried jointly by its rows, columns, headers, and the relationships among cells, so information is inevitably lost when a table is linearized for a model [2]. How a table is *represented* to the model is therefore a decisive variable; because this choice bears most directly on reasoning over content, we treat it within the data-understanding pillar (Section 2.2.1) rather than as a separate concern. Rather than cataloguing papers in isolation, we synthesize the literature thematically within each pillar, and we close each pillar by contrasting what current work achieves on academic benchmarks with what it still lacks for real-world, industry-scale deployment, the gap that motivates this report.

2.1 Schema Understanding

Schema understanding concerns the structural blueprint of tabular data (table and column names, data types, primary and foreign keys, and inter-table relationships) and the task of mapping a user's natural-language intent onto that structure. Its canonical instantiation is Text-to-SQL: translating a natural-language question into an executable query over a relational database. The schema itself is the model's input representation here, expressed for example as SQL `CREATE TABLE` statements or dataframe declarations that expose structure while omitting the bulk of the values; this sidesteps context-length limits but presupposes that the schema is well-formed and interpretable [2, 3]. On curated academic schemas Text-to-SQL is largely solved; the difficulty, and the reason the problem remains open, is *industry scale*. Deploying this capability in realistic settings is hard for three reasons: schemas are large (thousands of tables and tens of thousands of columns) and their naming is often cryptic, undocumented, and obsolescent, drifting as the database evolves; the relevant schema elements must be retrieved from among this mass before a query can be formed; and the generated query must be not merely syntactically

valid but logically faithful to the user's intent [8]. We organize the literature around how it addresses these three challenges.

2.1.1 Knowledge-base and metadata construction

The first challenge (obscure schemas) has motivated a line of work that enriches the raw schema with a semantic layer before any query is attempted. Such knowledge bases are built by aligning schema metadata with business glossaries [11], by automatically generating column and table descriptions [12], or by injecting domain knowledge from external documents [13]. These descriptions disambiguate cryptic identifiers (for example, resolving a column named `A2` to its business meaning) and externalize schema semantics into a structured artifact that a model can consult, which is essential when industry-scale schemas exceed the model's context window. A recurring limitation of this line of work is that it treats the knowledge base as a *static* asset, constructed once and thereafter fixed; it does not adapt as new queries reveal previously undocumented aspects of the schema.

2.1.2 Schema linking and retrieval

The second challenge (identifying which tables and columns are relevant to a given question) is a retrieval problem. Retrieval-Augmented Generation (RAG) [14] provides the general template of grounding generation in retrieved context, and dense retrievers built on sentence embeddings [15] make it possible to match a natural-language query against schema-element descriptions by semantic similarity. For databases, retrieval must respect structure: relevant tables imply related tables through foreign-key relationships, so join-aware and hierarchical retrieval (first selecting tables, then narrowing to columns, and expanding along relationships to preserve join paths) is more effective than flat retrieval. A central difficulty is that vague user queries map poorly onto structured schemas, which has prompted iterative approaches that reformulate the query when the retrieved context is judged insufficient, echoing self-reflective RAG methods developed for open-domain settings [16].

2.1.3 From encoder–decoder models to LLM pipelines

The third challenge (generating faithful queries) has driven the methodological evolution of the field. Early neural systems framed Text-to-SQL as sequence generation: Seq2SQL used reinforcement learning to generate structured queries [4], and SQLNet replaced generation with a sketch-based slot-filling formulation to avoid the pitfalls of ordering in the reward [17]. The introduction of large, cross-domain benchmarks such as Spider [18] and, more recently, the industry-scale BIRD benchmark [8] shifted evaluation toward generalization across unseen schemas and toward execution accuracy on realistic databases. With LLMs, the dominant paradigm became in-context learning: DIN-SQL decomposes the task into sub-problems and adds self-correction [5]; DAIL-SQL systematically studies prompt construction and exemplar selection [19]; SQLPrompt explores few-shot prompting for the task [20]; MAC-SQL casts generation as a collaboration among specialized agents (selector, decomposer, refiner) [6];

and CHASE-SQL generates multiple candidate queries via diverse reasoning paths and selects among them with a preference-optimized selector [7]. The clear trend is from single-pass generation toward multi-step pipelines that decompose, verify, and correct.

2.1.4 Self-reflection for structured generation

A unifying thread across the most recent methods is *self-reflection*: rather than trusting a single generation, the model scores its own output and iteratively refines it. The idea originates in open-domain generation, where self-reflection improves factuality and consistency by having a model critique and revise its answers [21], and where self-reflective retrieval decides when and what to retrieve [16]. Adapting this paradigm to Text-to-SQL is non-trivial because the domain offers concrete, checkable signals (a query can be executed, its syntax validated, and its results inspected) that open-domain generation lacks. Using an LLM as an evaluator (LLM-as-a-judge) makes it possible to score retrieval relevance, semantic correctness, and result–intent alignment, though such judges must be prompted carefully with rubrics and examples to mitigate the biases inherent in LLM-based evaluation [22]. Our own work, described in the next chapter, builds directly on this thread by integrating reflection at the retrieval, generation, and verification stages of a Text-to-SQL pipeline.

2.1.5 From benchmarks to the enterprise

Despite rapid progress, the schema-understanding literature is overwhelmingly validated on curated benchmarks such as Spider [18] and BIRD [8], whose schemas are modest in size, consistently named, and fully documented. Three gaps separate these reported results from real-world deployment. First, *scale and obscurity*: production schemas carry thousands of tables and cryptic, undocumented column names, so methods that assume the whole schema fits in context, or that rely on self-explanatory names, degrade sharply. Second, *obsolescence*: real schemas drift as columns are added, repurposed, or abandoned, yet most systems treat the schema, and any semantic layer built over it, as a static artifact fixed at setup time. Third, *the absence of gold queries*: benchmarks supply reference SQL for evaluation, whereas a deployed system must judge the correctness of its own output with no reference to compare against. These gaps motivate the self-reflective, continually-enriched design we present in Chapter 3.

2.2 Data Understanding

Data understanding concerns reasoning over the *content* of tables (their values and the aggregations, comparisons, trends, and outliers they encode), frequently in combination with surrounding text. Whereas schema understanding privileges the structural blueprint, data understanding privileges the cells themselves and the semantics that connect them. Its canonical instantiation, and the task we concentrate on, is Table Question Answering. We first review how table content is represented to the model, then treat Table QA as the representative task (noting two adjacent tasks for context), and finally consolidate the open challenges and the real-world gap that define our forward direction.

2.2.1 Representing table content

Before a model can reason over a table, its two-dimensional content must be rendered into a form the model consumes, and performance is sensitive to this choice, varying by several percentage points with the format [3]. Three families present content directly, each trading completeness against compactness differently; the schema-based view of Section 2.1 abstracts structure rather than content and belongs with that pillar. *Serialization* renders the table as text (HTML, Markdown, JSON, LaTeX, or delimited variants); it is the default in most benchmarks because such formats occur naturally in pretraining data, but long or multi-table inputs quickly exceed the context window, forcing truncation or row- and column-sampling [2]. *Image* representations render the table visually, preserving merged cells and hierarchical headers that serialization distorts, at the cost of resolution-bounded legibility as tables grow [2]. *Table encoders* embed two-dimensional structure directly into the model; table foundation models such as TableGPT2 couple a row- and column-aware encoder with a large base model and outperform generalist prompting across a range of tasks [23]. A limitation common to all three families, and consequential for deployment, is that they are demonstrated on small tables, typically a handful of columns and a few dozen rows, and none scales gracefully to the hundreds of columns and thousands to millions of rows of real-world data.

2.2.2 Graph representations

A distinct and, we argue, especially promising way to represent a table casts it as a *graph*: cells, rows, headers, and their groupings become typed nodes, and the table's relational topology (which cell lies in which row and column, how headers span sub-columns, and how foreign keys link tables) becomes edges. Unlike the linear formats above, a graph makes a table's structure explicit and *addressable* rather than implicit in a flattened token stream, so a model can navigate directly to the region that matters instead of scanning the entire serialization. This composes naturally with graph-based retrieval, which delivers only the relevant subgraph to the model: GraphRAG builds a graph index for query-focused retrieval [24], G-Retriever retrieves relevant subgraphs for question answering over textual graphs [25], and agentic navigators such as GraphOTTER walk a graph of a complex table from headers to the cells that answer a query [26]. For hierarchical and relationally complex tables, this structure-aware selection is a natural fit for Table Question Answering, where the answer-bearing cells must be located before any reasoning can occur.

The promise, however, is only partly realized. Current graph methods are confined either to *document-* and text-graph settings, where GraphRAG and G-Retriever operate over prose rather than dense tabular data, or to *small, complex* tables, where GraphOTTER demonstrates graph reasoning but scores every traversal step with the LLM, tying latency to table size. Graph representation is therefore a solid foundation whose use today is limited to documentation-style graphs and small-scale tables. The motivation we carry forward is to lift this ceiling: to represent *large* tabular data as a graph and to retrieve over it by addressing pre-computed structure rather than by LLM-scored traversal, so that the

structure-awareness that already benefits Table QA on small tables extends to the hundreds of columns and thousands of rows of real-world data. We develop this direction in Chapter 4.

2.2.3 Table Question Answering

Table Question Answering (TQA) is the most studied task: given a natural-language question and one or more tables, sometimes accompanied by a passage, the model produces an answer derived from the table content, typically as cell spans, computed values, or short text spans [2]. The benchmark landscape has expanded along several axes. Foundational datasets such as WikiTableQuestions [27] and the sequential SQA [28] established open-domain answering over web tables. Subsequent benchmarks pushed on *domain* (finance in TAT-QA [29] and FinQA [30]), on *input complexity* (hybrid tabular–textual reasoning in HybridQA [31], hierarchical tables in HiTab [9], and multi-hierarchical and multi-table settings in MULTIHIERTT [10] and MultiTabQA [32]), and on *answer format* (free-form generation in FeTaQA [33]). A notable methodological finding is that decomposition-based pipelines, which break a question into simpler steps executed against the table, have driven large gains and now approach saturation on the simpler retrieval-oriented benchmarks. A caveat qualifies these gains: the tables in these benchmarks are small, and the decomposition pipelines that drive the gains issue an LLM call per step, so both the evaluation and the method strain once tables reach industry scale, where the answer-bearing cells are a vanishing fraction of the content. This foreshadows the robustness and efficiency challenges discussed below.

2.2.4 Adjacent tasks

Two further tasks round out the data-understanding pillar but are not our focus. *Table Fact Verification* (TFV), also called table-based natural language inference, asks whether a claim is supported, refuted, or unverifiable given a reference table; TabFact established the task at scale over Wikipedia tables [34], and modern methods exceed 80% accuracy, though the strong numbers concentrate on well-structured single tables and thus test fact retrieval more than deep reasoning. *Table-to-Text* generation instead asks a model to describe table content faithfully in natural language, ranging from controlled descriptions of highlighted cells (ToTTo [35]) to query-focused summaries that must reason about underlying causes rather than surface content (QTSUMM [36]). Both tasks share Table Question Answering's core difficulty, isolating and reasoning over the relevant cells, which is why we take Table QA as the representative task for the pillar.

2.2.5 Open challenges

Synthesizing across tasks, the survey literature identifies three challenges that define the frontier of data understanding [2].

First, limited scope beyond retrieval and arithmetic. Many benchmarks are constructed by translating SQL queries or mathematical expressions into natural-language questions, so their core structure

rewards retrieval and computation rather than higher-order reasoning. As several such benchmarks approach saturation, recent work has begun to demand diagnostic, predictive, and prescriptive thinking (identifying insights, forecasting, and generating charts from ambiguous requests) and intent-driven querying, where a system must infer what an abstract, goal-oriented request actually asks for [37, 38]. These tasks cannot be solved by matching synonyms to table headers; they require understanding the table's overall context and the user's intent.

Second, lack of robustness on complex inputs. Intricate table structures, long tables, lengthy accompanying texts, and multi-table settings degrade model performance far more than they degrade human performance. On hierarchical and multi-table benchmarks such as HiTab [9] and MULTI-HIERTT [10], reported accuracies fall below 50% against human performance around 83%, and even on well-structured multi-table QA the strongest evaluated models reach only about half of human accuracy. Robustness to input complexity is thus a first-order open problem.

Third, limited generalization across representations. As noted in Section 2.2.1, performance varies by up to several percentage points with the input format, and models struggle to transfer across the diverse representations used by different benchmarks [3]. Proposed remedies include standardizing serialization options, deriving serialized views from images of complex tables, and training on serialization-to-serialization tasks that convert one format into another to build robustness to varied input styles.

2.2.6 From benchmarks to the enterprise

These challenges share a root cause that parallels the schema pillar: data-understanding methods are developed and scored on small, clean, well-structured tables, whereas real applications demand reasoning over data that is large, noisy, hierarchical, and spread across many tables. The field is near saturation on simple retrieval-oriented benchmarks yet remains far below human performance the moment inputs resemble production data, and the pipelines that post the strongest numbers scale their cost with table size, so they do not transfer to tables that cannot fit in a model's context. Reasoning faithfully over content at real-world scale and complexity, rather than posting incremental gains on already-saturated simple benchmarks, is the frontier this report takes as its forward direction.

Having mapped both pillars and the real-world gaps that remain, the next chapter turns to our own contribution to the schema-understanding pillar, Reflect-SQL, which operationalizes this analysis and lays the groundwork for pursuing data understanding at industry scale.

Chapter 3

Current Progress: Reflect-SQL

Translating natural language into SQL is nearly solved on curated academic benchmarks, yet the same systems falter on the databases that matter in practice. Three limitations of current approaches explain the gap, and together they define what a system built for industry scale must overcome. First, most LLM pipelines generate a query in a *single pass*, or with only shallow self-correction, so they have no mechanism to recover when a large, unfamiliar schema leads them astray. Second, they implicitly assume the schema is *small enough to fit in context and self-explanatory*, whereas enterprise schemas span thousands of tables and tens of thousands of cryptically named columns that no model can interpret unaided, and that drift and go stale as the business evolves. Third, they are tuned and evaluated against *gold queries*, a luxury that does not exist in production, where the system itself must decide whether a generated query is correct before returning it. Meeting these conditions at industry scale calls for treating schema understanding, retrieval, generation, and validation not as a single forward pass but as fallible steps to be checked and repaired.

This chapter presents our research contribution to date, *Reflect-SQL* [1], a framework for Text-to-SQL built on exactly this principle and operationalizing the schema-understanding pillar developed in Chapter 2. As established there, deploying Text-to-SQL in such realistic settings is hindered by three systemic difficulties that mirror the limitations above: (i) **schema obscurity**, the large-scale databases with cryptic column names (e.g., `A2`, `dname`) and undocumented, drifting relationships; (ii) **contextual retrieval**, the difficulty of extracting the relevant schema elements, from among thousands of candidates, when vague user queries map poorly onto structured schemas; and (iii) **logical integrity**, the need to move beyond syntactic validity to guarantee that a query faithfully answers the user's intent. Reflect-SQL addresses these through a multi-stage *self-reflection* approach that, rather than trusting a single-pass generation, scores its own intermediate outputs and iteratively refines them within interconnected feedback loops. This directly extends the self-reflection thread of Section 2.1 to the structured domain of databases, exploiting the concrete, checkable signals (executability, syntax, and result inspection) that this domain uniquely affords.

The primary contributions of the framework are: (1) a complete, end-to-end Text-to-SQL system for enterprise settings; (2) a **dynamic knowledge base** that enriches schema understanding over successful executions; (3) a **reference-free** methodology for validating SQL without gold-standard labels;

and (4) a **feedback-driven query refinement** process that aligns ambiguous user queries with the database schema. On the challenging BIRD benchmark [8], Reflect-SQL attains an execution accuracy of 72.03%, substantially outperforming strong baselines.

3.1 Framework Overview

The framework is organized as a knowledge base feeding three interconnected reflective loops (retrieval, SQL generation, and entailment) as depicted in Fig. 3.1. A dynamic **knowledge base** (KB) acts as a semantic abstraction layer over the raw schema. A **feedback-driven retrieval** loop then identifies the tables and columns relevant to a query, an **iterative SQL synthesis** loop generates and corrects a candidate query, and a final **entailment** check verifies that the executed result matches the user's intent before returning it and enriching the KB.

Each component is shaped by one of the industry-scale pressures identified above, and this is the lens through which the rest of the chapter should be read. The **knowledge base** answers schema obscurity and context limits: rather than feeding the model a raw schema it cannot fit or interpret, the KB externalizes a semantic layer, descriptions, metadata, and relationships, that the model consults on demand. Its **iterative refinement** answers schema drift: because an enterprise schema is never fully documented and never stops changing, the KB cannot be built once and frozen, so every validated query feeds its result back to enrich descriptions and relationships, turning deployment into continual learning rather than a one-time setup. The **SQL synthesis** loop answers the fragility of single-pass generation, catching and correcting syntactic and semantic faults before a query is ever executed. And **results validation**, through the entailment check, answers the absence of gold labels in production: it judges the executed result against the user's intent in a reference-free manner, so the system can decide for itself whether an answer is trustworthy. We detail each component in turn.

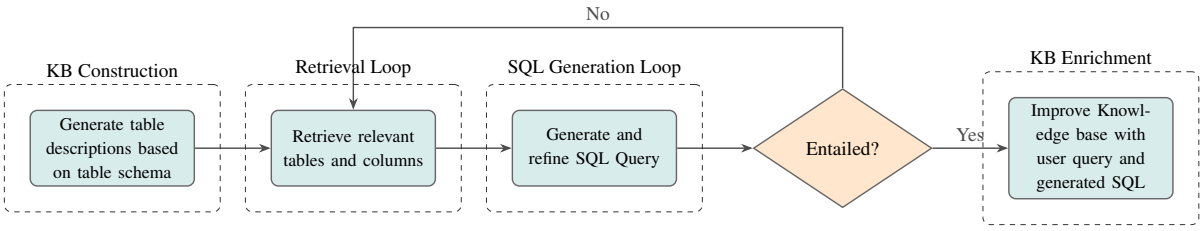


Figure 3.1 High-level architecture of the Reflect-SQL framework.

3.2 Knowledge Base Construction and Refinement

The knowledge base is a semantic abstraction layer containing table descriptions, column metadata, and relationships. It is vital for industry-scale databases, which routinely exceed the context limits of LLMs and suffer from obscure naming conventions. By externalizing schema details into a struc-

tured KB, the framework overcomes context constraints and limited schema understanding, ensuring the LLM accesses only the specific information needed to bridge user intent and the database. This design responds directly to the static-KB limitation identified in Section 2.1: the KB here is not a one-time artifact but is continuously refined.

3.2.1 Construction

For the BIRD dataset [8], the KB is constructed by synthesizing the database schema, sample data values, and the provided training questions and external evidence. In broader enterprise settings, the same process ingests domain knowledge from business documents and glossaries. These inputs are processed to generate comprehensive descriptions for tables, columns, and their relationships. To enable efficient retrieval, the components are encoded into dense vector representations using a sentence embedding model [15]. The vector database is structured hierarchically: a global collection for table descriptions and separate local collections for each table's column descriptions. This supports a two-step retrieval process: first identifying relevant tables, then narrowing to specific columns. A representative JSON structure of the KB is shown in Fig. 3.2.

```
{
  "financial": {
    "account": {
      "table_description": "it stores information about bank accounts ...",
      "relationships": [{
          "column_name": "district_id", "related_table": "district",
          "related_column": "district_id" }...],
      "column_description": {
        "account_id": "Unique identifier for each bank account...",
        "district_id": "Identifier representing the district...",
        ...}
    }...}
  ...
}
```

Figure 3.2 Example JSON specification of the knowledge base.

3.2.2 Iterative refinement

In a post-processing step, after a successful SQL execution, one that passes syntactic and semantic validation and whose entailment score exceeds a threshold θ_e, the knowledge base is refined. This conditional update is critical for preventing error propagation: low-quality or contextually incorrect queries could otherwise pollute the KB, so only validated queries with validated results contribute to the system's learning. This mechanism is essential in real-world environments with ambiguous column names and continuously evolving schemas. Table 3.1 illustrates how column descriptions are enriched after a successful query.

Component	Before Refinement	After Refinement
`loan.status`	Categorical column for loan status with values 'A', 'B', 'C', 'D'.	Loan repayment state: 'A' indicates finished or paid, 'B' indicates defaulted...
`trans.k_symbol`	Text column named 'k_symbol' with values 'POJISTNE', 'SIPO'.	Transaction Category: determines the payment type (e.g. 'POJISTNE' maps to Insurance)...

Table 3.1 Example of knowledge base refinement for BIRD [8].

3.3 Iterative RAG Retrieval

In the retrieval phase, the curated KB is used to identify the tables and columns relevant to a user query, as detailed in Fig. 3.3.

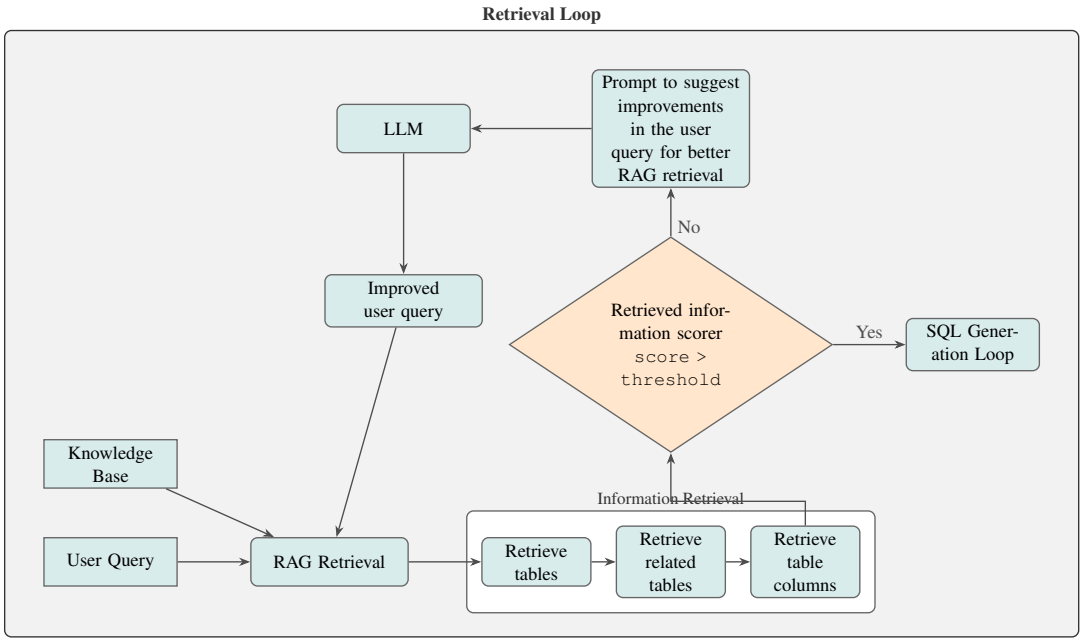


Figure 3.3 Iterative retrieval loop.

3.3.1 Hierarchical table and column retrieval

Given a user query Q, we embed it and use cosine similarity (Eq. 3.1) to retrieve the top-k tables $\{T_1, \ldots, T_k\}$ from the vector database.

$$\text{score}(T_i, Q) = \cos(\vec{t_i}, \vec{q}) = \frac{\vec{t_i} \cdot \vec{q}}{\|\vec{t_i}\| \|\vec{q}\|} \tag{3.1}$$

This set is expanded by including tables related via relationships defined in the KB, ensuring join paths are available, the join-aware retrieval principle discussed in Section 2.1. For each selected table, a similar embedding-based search retrieves the most relevant columns.

3.3.2 Relevance and completeness scoring

We assess retrieval quality with two metrics: a **relevance score** (R) for semantic alignment and a **completeness score** (C) to ensure all necessary schema elements are present. These are produced by an LLM-as-a-judge, and the final retrieval score $Score_r$ is a convex combination (Eq. 3.2).

$$Score_r = \alpha R + (1 - \alpha)C, \quad \text{where} \quad \alpha \in [0, 1] \tag{3.2}$$

3.3.3 Query refinement

If the retrieved schema is insufficient ($Score_r < \theta_r$), an iterative refinement loop is triggered. In each iteration, the LLM reformulates the user query into a more detailed version Q' to sharpen its semantic focus, while the retrieval scope is simultaneously widened by increasing the number of tables and columns retrieved in the next pass. Refining the query's focus while broadening the search area lets the system converge on the complete set of schema elements required for complex queries. The loop terminates once $Score_r \geq \theta_r$ or a maximum number of iterations is reached.

3.4 SQL Query Generation

With a high-quality set of retrieved tables, columns, descriptions, and sample data, a candidate SQL query is generated by an LLM and then subjected to two nested validation loops, as shown in Fig. 3.4.

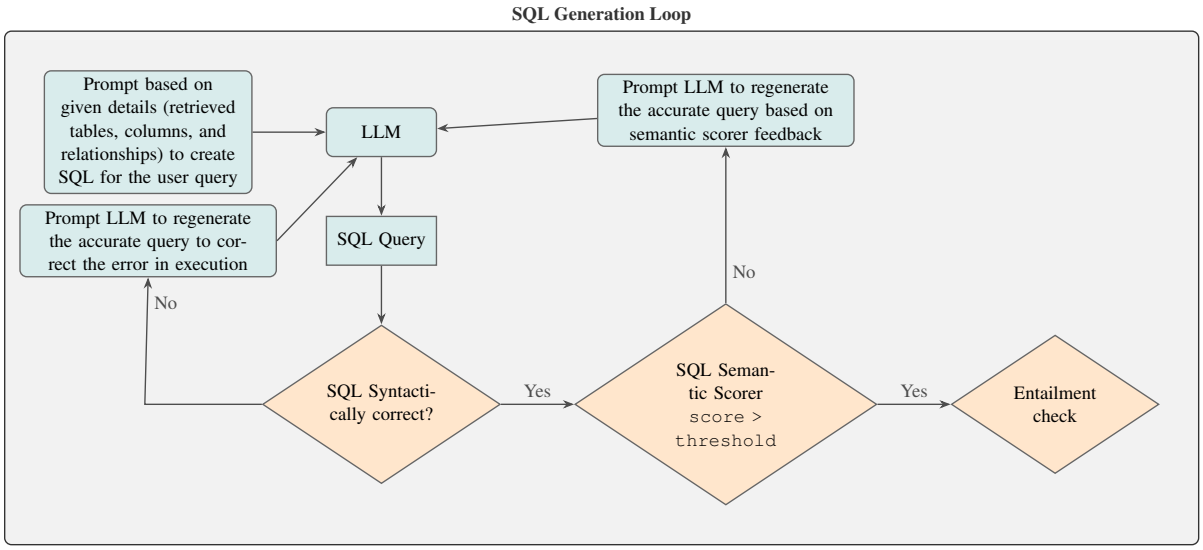


Figure 3.4 Iterative SQL generation loop.

3.4.1 Syntactic validation and correction

Each generated query is immediately checked for syntactic validity using a database engine. On a syntax error, a correction loop feeds the original query, the error message, and the schema context back to the LLM with instructions to fix the error. The loop repeats until a syntactically correct query is produced or a maximum iteration limit is met, making the system resilient to basic generation mistakes.

3.4.2 Semantic validation

A syntactically valid query undergoes semantic validation across four dimensions: column-name consistency (C), granularity and aggregation (G), join and relationship consistency (J), and query-logic alignment (Q). An LLM-as-a-judge scores each dimension, and a final semantic score S is a weighted sum (Eq. 3.3).

$$S = w_c \cdot C + w_g \cdot G + w_j \cdot J + w_q \cdot Q \tag{3.3}$$

where w_c, w_g, w_j, w_q are predefined weights. If $S < \theta_s$, the query is returned to the LLM with diagnostic feedback for iterative correction.

3.5 Entailment

Once a query passes all validation, it is executed to retrieve the result set R. In a final verification stage, the framework checks whether R logically entails the user's original query Q, the reference-free validation that distinguishes this pillar's notion of *logical integrity* from mere syntactic correctness.

3.5.1 Entailment scoring

An entailment score $E(Q, R)$ is computed by an LLM-as-a-judge, assessing both **schema consistency** (E_{schema}) and **data consistency** (E_{data}), as in Eq. 3.4.

$$E(Q, R) = \beta \cdot E_{\text{schema}} + (1 - \beta) \cdot E_{\text{data}} \quad \text{where} \quad \beta \in [0, 1] \tag{3.4}$$

3.5.2 Entailment decision

If $E(Q, R) \geq \theta_e$, the result is returned to the user and the KB is updated with the successful query–result pair. Otherwise, the entire pipeline is re-triggered with feedback on the failure reason (e.g., missing columns, incorrect filters). This final loop ensures the output truly matches the user's intent.

3.6 Mitigating Bias in LLM-based Evaluation

Using an LLM as a judge can introduce biases [22]. To mitigate this, the framework employs a structured, few-shot prompting strategy for all scoring tasks: prompts provide detailed instructions, scoring

rubrics, and few-shot examples that anchor the LLM's judgment to task-aligned criteria, increasing the reliability and reproducibility of the evaluation while reducing the influence of the model's inherent biases.

3.7 Model Parameter Optimization

A systematic grid search determined the optimal thresholds ($\theta_r, \theta_s, \theta_e$) and weights ($\alpha, \beta, w_c, w_g, w_j, w_q$). Each parameter was varied and the combination yielding the highest validation accuracy was selected; the final values are given in Table 3.2.

Parameter	Value
θ_r	4
θ_s	5
θ_e	3
α	0.6
β	0.2
w_c	0.1
w_g	0.3
w_j	0.4
w_q	0.2

Table 3.2 Optimized parameters using grid search.

3.8 Evaluation

3.8.1 Experimental setup

We evaluate the framework on the **BIRD benchmark** [8]. Because large-scale, proprietary enterprise datasets are generally inaccessible for academic experiments, BIRD serves as an excellent proxy: it tests Text-to-SQL systems over diverse relational databases that mirror real-world complexity.

3.8.2 Results

We evaluate Claude, OpenAI, and DeepSeek API-based models, as well as an open-source DeepSeek model. Table 3.3 reports execution accuracy on BIRD. Across every model, enabling the reflection framework produces substantial gains over the corresponding non-reflective baseline (a 6–8% overall improvement), confirming that iterative reasoning and refinement are central to performance.

Model	Simple	Moderate	Challenging	Overall
Claude Models				
Claude-sonnet-4.5 (with reflection)	**88.97**	**50.65**	**32.41**	**72.03**
`Claude-sonnet-4.5 (w/o reflection)`	81.18	45.25	24.13	64.92
OpenAI API Models				
o1-preview (with reflection)	**75.89**	**48.49**	**18.62**	**62.19**
`o1-preview (w/o reflection)`	69.35	40.54	15.86	55.66
DeepSeek API Models				
deepseek-reasoner (with reflection)	**73.62**	**46.55**	**17.24**	**60.10**
`deepseek-reasoner` (w/o reflection)	62.90	40.54	12.41	51.88
Open Source Models (Local setup)				
DeepSeek-Coder-V2-Lite... (with reflection)	**43.55**	**21.62**	**3.44**	**35.02**
`DeepSeek-Coder-V2-Lite...` (w/o reflection)	30.65	13.51	0.0	22.64

Table 3.3 Accuracy (%) of various models on the BIRD dataset. Top performers in each category are highlighted.

3.8.3 Human evaluation

To complement the automatic evaluation, we conducted a qualitative human assessment of the correctness and semantic alignment of generated queries, focusing on the impact of the entailment check on subtle logical errors that syntax or basic semantic validation may miss. Table 3.4 shows a representative case in which entailment feedback corrects a division-by-zero and casting error that would otherwise return null results.

Stage	Analysis / SQL Snippet
User Query	List the lowest three eligible free rates for students in continuation schools.
Initial SQL	`SELECT ... (f.Count / f.Enrollment) ... WHERE s.SOCType LIKE '%Continuation%'`
Entailment Check Result	**Score**: 2 / 5 **Reason:** The result returns None values due to division by zero or nulls. Filters for positive enrollment and float casting are needed.
Refined SQL	`SELECT ... (CAST(f.Count AS REAL) / f.Enrollment) ... WHERE f.Enrollment > 0 AND s.SOCType LIKE '%Continuation%'`

Table 3.4 Example of SQL refinement driven by entailment feedback.

3.8.4 Comparison with state of the art

We benchmark against leading methods on the BIRD development set (Table 3.5). Reflect-SQL achieves an execution accuracy of 72.03%, a marked improvement over DIN-SQL [5] (50.72%), DAIL-

SQL [19] (57.41%), and MAC-SQL [6] (59.59%), and is comparable to the recent CHASE-SQL architecture [7], establishing Reflect-SQL as a top-tier solution for complex database querying.

Method	Overall Accuracy
BIRD paper [8]	46.35
DIN-SQL [5]	50.72
DAIL-SQL [19]	57.41
MAC-SQL [6]	59.59
CHASE-SQL + Claude 3.5 Sonnet [7]	69.53
CHASE-SQL + Gemini 1.5 Pro [7]	**73.01**
Reflect-SQL + Claude 4.5 Sonnet (Ours)	**72.03**

Table 3.5 Comparison with state-of-the-art methods on the BIRD dev set (execution accuracy %).

3.9 Ablation Study

To quantify each component's contribution, we performed an ablation using `deepseek-chat` as the base model (Table 3.6). Removing any component degrades performance, and the self-reflection loop in SQL generation proves most critical: its removal causes the sharpest drop. These results affirm the effectiveness of the multi-stage, reflective design.

Variant	Simple	Moderate	Challenging	Overall
Full Model (Self-Reflect)	68.64	43.10	14.48	55.80
w/o Self-Reflection in SQL Generation	62.91	37.93	12.41	50.58
w/o Iterative Retrieval	67.02	40.30	10.34	53.58
w/o Entailment Check	64.10	41.59	13.10	52.47

Table 3.6 Ablation results showing the contribution of each module.

3.10 Summary

Reflect-SQL demonstrates that self-reflection, adapted to the structured domain of databases, is an effective route to reliable schema understanding. Its dynamic knowledge base disambiguates obscure schemas and adapts over time; its feedback-driven retrieval aligns vague queries with the schema; and its layered generation and entailment loops enforce logical integrity without gold-standard labels. The framework's effectiveness depends on the initial quality of the knowledge base and can face scalability challenges with extremely large schemas, and the automatic metrics for semantic correctness and entailment remain an evolving area of research. These observations, together with the data-understanding challenges of Chapter 2, motivate the research direction discussed in the concluding chapter.

Chapter 4

Future Directions

The preceding chapters traced tabular data understanding across two pillars (schema understanding and data understanding) and presented our contribution, Reflect-SQL [1], to the former. This chapter sets out the research agenda that follows from that work. One concern unifies the entire agenda: making both pillars work at *industry scale*, the regime of large, obsolescent schemas and tables too big for a model's context, where the methods that succeed on academic benchmarks break down. It is deliberately extensive and organized in two parts. The first part consolidates what the schema-understanding pillar has achieved and identifies the problems that remain open, since these directly extend our completed and ongoing work. The second part charts the direction we intend to pursue next: the data-understanding pillar, where the survey's open challenges (Section 2.2) define the frontier and where representing tabular data at scale, concretely as a graph, is the organizing commitment from which evidence retrieval and the remaining challenges follow.

4.1 Future Directions in Schema Understanding

The schema-understanding pillar has matured along the three axes identified in Section 2.1. Obscure schemas are now routinely enriched with a semantic layer through knowledge-base and metadata construction; relevant schema elements are located through join-aware, iterative retrieval; and faithful queries are produced by multi-step pipelines that decompose, verify, and correct rather than generate in a single pass. Reflect-SQL advances this frontier by threading self-reflection through all three stages (knowledge-base construction, retrieval, and synthesis) and grounding it in the checkable signals that the SQL setting affords, reaching 72.03% execution accuracy on BIRD. Yet several problems remain unsolved, and they define the near-term agenda.

4.1.1 From static to continually evolving knowledge bases

A recurring limitation across the knowledge-base literature, and a constraint our own work shares, is that the semantic layer is treated as a *static* artifact: it is constructed once and thereafter fixed. Real schemas, however, drift: columns are added, business meanings shift, and each new query may reveal an undocumented facet of the data. The open problem is a *continually evolving* knowledge base that

updates itself from query logs, execution feedback, and user corrections, versioning schema semantics as they change rather than assuming a one-time build. Such a knowledge base would turn schema understanding from a preprocessing step into a lifelong-learning loop, in which every resolved query improves the system's model of the schema for the next one.

4.1.2 Retrieval at industry scale

Schema linking is accurate on academic benchmarks but becomes the binding constraint at industry scale, where a database may hold thousands of tables and tens of thousands of columns that cannot fit in a model's context window. The iterative, reflection-driven retrieval that Reflect-SQL employs is effective precisely because it reformulates the query when the retrieved context is judged insufficient, but every reformulation is another LLM call, so retrieval cost grows with the difficulty of the query. This mirrors a tension observed more broadly in structured retrieval, where coupling selection decisions to LLM invocations makes precision expensive [39,40]. A promising direction is to reframe schema retrieval from a query-time *search* into addressing against *pre-computed* structure: materializing foreign-key graphs, column-value indices, and description embeddings offline so that schema linking navigates directly to the relevant elements at near-constant cost, reserving LLM calls for genuine disambiguation rather than routine traversal. Decoupling retrieval from repeated model invocation is, in our view, the key to schema understanding that scales.

4.1.3 Cross-database and multi-table join reasoning

Reflect-SQL, like most current systems, is evaluated over a single database at a time. Enterprise text-to-SQL workflows are considerably harder: they span multiple databases and dialects, require long foreign-key chains and deeply nested queries, and demand reasoning about which of many near-duplicate tables is authoritative [37]. Extending self-reflective synthesis to this setting, where the space of join paths is combinatorial and a single wrong join silently corrupts the result, is an open problem. Join-aware retrieval that preserves relationship structure (Section 2.1) is a necessary but not sufficient ingredient; the harder question is how reflection should reason about join *correctness* when execution alone cannot distinguish a plausible-but-wrong join from the intended one.

4.1.4 Reliable, reference-free verification

Reflect-SQL's central bet is that reflection can be grounded in checkable signals (syntax, execution, and result–intent entailment scored by an LLM judge) so that queries can be validated without a gold reference. The reliability of that judgment is itself an open problem. LLM-as-a-judge is known to carry position, verbosity, and self-preference biases [22], and a judge that is confidently wrong is more dangerous than one that abstains. Future work should pursue calibrated, reference-free verification: quantifying when a self-assessment can be trusted, learning when to escalate to a human or to additional

evidence rather than accepting a flawed query, and stress-testing the semantic-validation dimensions against adversarial and near-miss queries where syntactic validity and semantic faithfulness diverge.

4.1.5 The cost of reflection

Reflection buys accuracy at the price of latency: iterative retrieval and iterative synthesis, each followed by multi-dimensional validation, multiply the number of LLM calls per query. For interactive analytics this cost is a deployment barrier. The open question is how to preserve the accuracy gains of reflection while collapsing its cost, for instance by distilling multi-step reflection into fewer passes, caching validated sub-queries and retrieval decisions across a session, and learning a stopping policy that halts refinement once marginal gains vanish. Efficiency here is not a secondary concern but a first-class research problem, and it foreshadows the efficiency–accuracy tension that dominates the data-understanding pillar below.

4.1.6 Beyond SQL

Finally, schema understanding is broader than SQL generation. Real analytical requests are frequently abstract and goal-oriented ("why did revenue dip last quarter?") rather than a clean specification of a query, and answering them may call for dataframe or general code generation rather than a single SQL statement [37,38]. Inferring the underlying analytical intent, and mapping it onto whichever execution substrate fits, generalizes the schema-linking problem and forms a natural bridge to the data-understanding pillar, to which we now turn.

4.2 Setting a Direction for Data Understanding

Having established schema understanding through Reflect-SQL, our next research phase turns to the second pillar: data understanding, and concretely Table Question Answering, the problem of reasoning faithfully over the *content* of tables. The survey literature frames this frontier through three challenges (Section 2.2): limited scope beyond retrieval and arithmetic, a lack of robustness on complex inputs, and limited generalization across representations. These challenges tell us *what* is missing; the purpose of this section is to identify *where* to begin. Our analysis points to a single organizing commitment from which the rest follow: *how a large table is represented to the model*. The representations in use today are demonstrated almost exclusively on small, clean tables (Section 2.2.1) and none scales to the hundreds of columns and thousands-to-millions of rows of real data; we therefore take *representing tabular data at scale*, concretely as a graph, as the lead direction, and treat evidence retrieval, robustness, and higher-order reasoning as problems this representation is designed to unlock. We close by arguing that progress on all of them is gated by a benchmark that does not yet exist: Table Question Answering at industry scale.

4.2.1 Representing large-scale tabular data as a graph

The representations surveyed in Section 2.2.1 all flatten a table into a token stream (serialization), a pixel grid (image), or a fixed-width embedding (table encoders), and all are validated on tables small enough to fit whole in a model's input. None survives contact with industry-scale data: a table of hundreds of columns and millions of rows cannot be serialized within any context window, rendered legibly as an image, or embedded without discarding most of its content. The graph representation of Section 2.2.2 is, in our view, the most promising escape from this ceiling, but it has so far been demonstrated only on document graphs and small, complex tables. The direction we intend to pursue is to lift it to scale: to represent a large table as a graph in which cells, rows, columns, headers, and their groupings are typed nodes, and the table's relational topology (row–column membership, header spans, and foreign-key links across tables) forms the edges, materialized once and offline rather than reconstructed per query. Such a graph makes a table's structure explicit and *addressable*: rather than asking a model to locate information inside a flattened dump, the structure itself records where everything is, so reaching a region of interest becomes a lookup rather than a scan. Making this representation hold at scale (how to construct the graph incrementally, index it for sub-linear access, and keep it current as the table evolves) is the central problem of this pillar, and the one on which the directions that follow depend.

4.2.2 Evidence retrieval as addressing over the graph

Given such a representation, evidence retrieval (placing the handful of answer-bearing cells in front of the model before it reasons) changes character. Today it is the binding constraint: as tables grow to hundreds of rows and columns, an LLM must locate a few relevant cells amid overwhelmingly irrelevant content, and both context budget and accuracy degrade sharply as noise grows, the "lost-in-the-middle" effect [41]. The deeper difficulty is that prevailing approaches couple cell selection to LLM invocation: full-table linearization performs no selection and exhausts the budget on large inputs; decomposition pipelines such as Chain-of-Table [40] and DATER [39] select rows and columns through a sequence of LLM calls, so every filtering decision costs an invocation and errors compound across stages; and even graph-based navigators such as GraphOTTER [26] traverse a structured table but score each hop with the LLM, tying latency to table size. In every case improving precision demands more LLM calls while reducing them risks discarding evidence, so an efficiency–accuracy tension appears fundamental. A pre-computed graph dissolves this tension by construction: because the groupings, statistics, and semantic partitions needed to navigate are query-independent, they can be computed offline, and retrieval becomes *addressing* against that structure (navigating directly to the intersection of the relevant rows and columns) rather than an LLM-scored search, reserving the model for final synthesis. This decoupling of selection from reasoning is the direct content-side analogue of the schema-retrieval direction of Section 4.1, and unifying the two under a single "retrieval-as-addressing" principle is the through-line we intend to develop across both pillars.

4.2.3 Robustness on complex table structures

The efficiency bottleneck is most acute precisely where the survey reports the largest human–model gaps: hierarchical, long, and multi-table inputs, where accuracies fall below 50% against human performance around 83% on benchmarks such as HiTab [9] and MULTIHIERTT [10]. Complex structure compounds the retrieval problem: multi-row headers carry span labels that flat serialization loses, and merged regions create implicit repetition that corrupts naive cell addressing. A robust system must therefore condition the table before reasoning (propagating hierarchical headers and normalizing merged cells) and carry that structure through to the representation, whether via specialized encoders in the tradition of TAPAS [42] and TaBERT [43], via a graph that encodes multi-row headers and merged regions as typed nodes and edges, or via structure-aware indices built over serialized views. Robustness to input complexity, rather than incremental gains on already-saturated simple benchmarks, is where we intend to concentrate.

4.2.4 Reasoning beyond retrieval and arithmetic

Once evidence can be delivered reliably, the harder question is what the model does with it. Many benchmarks are constructed by translating SQL or arithmetic expressions into natural language, so they reward matching synonyms to headers rather than genuine reasoning, and they are approaching saturation. The frontier is diagnostic, predictive, and prescriptive analysis (identifying insights, forecasting, and responding to abstract, goal-oriented requests) which requires understanding the table's overall context and the user's intent rather than surface retrieval [37, 38]. This is also where the two pillars converge: inferring analytical intent (a schema-understanding capability) and reasoning over content (a data-understanding capability) are two halves of the same problem, and a system that first understands structure and then reasons over content is the long-term target.

4.2.5 Generalization across representations

Finally, model performance varies by several percentage points with the input format, and systems transfer poorly across the diverse representations used by different benchmarks (Section 2.2.1) [3]. Promising remedies include standardizing serialization options, deriving serialized views from images of complex tables, and training on serialization-to-serialization tasks that convert one format into another to build robustness. Any representation we develop should be evaluated for representation-invariance from the outset, so that gains do not prove brittle to a change of input format.

4.2.6 A large-scale benchmark for Table Question Answering

A direction that conditions all of the above is measurement. The benchmarks that define the field (WikiTableQuestions, HiTab, MULTIHIERTT, and the others surveyed in Section 2.2) are built on tables of at most a few dozen rows and columns, and the retrieval-oriented ones are already approaching saturation; none exercises Table Question Answering at the scale where our representation and retrieval

claims would actually be tested. There is, in short, no analogue of what BIRD [8] provided for Text-to-SQL: an industry-scale yardstick. Constructing one (tables with hundreds of columns and thousands-to-millions of rows, hierarchical and multi-table structure, realistic noise and obsolescent headers, and questions whose answers hinge on a vanishing fraction of the cells) is therefore a prerequisite rather than an afterthought. Such a benchmark would make the efficiency–accuracy trade-off measurable (cost per query against accuracy as table size grows), expose the brittleness that small, clean tables hide, and give the representation-at-scale and addressing directions above a concrete target against which progress can be judged. We regard building or curating this benchmark as part of the work, not a precondition to be supplied by others.

4.3 Summary

Taken together, these directions describe a coherent trajectory. In schema understanding, the immediate work is to make the semantic layer continually evolving, to scale retrieval by addressing rather than search, to extend faithful synthesis to cross-database joins, and to make reflection both reliable and cheap. In data understanding, our next pillar, the lead direction is to represent large tabular data at scale, concretely as a graph, so that evidence retrieval becomes addressing rather than search; robustness on complex structures, higher-order reasoning, and representation-invariance follow as targets this representation is meant to unlock, and a large-scale Table Question Answering benchmark is the instrument needed to measure progress on all of them. The unifying thread is that reliable tabular reasoning depends less on ever-larger models than on the architecture that delivers the right structure and the right evidence to them.

Chapter 5

Conclusion

This report has examined *tabular data understanding* with large language models through two complementary pillars (schema understanding and data understanding) and situated our own contribution within them.

Chapter 2 framed the field around these two pillars. It began from the representation mismatch that makes tabular reasoning hard: because language models consume one-dimensional token sequences while tables are inherently two-dimensional, every pipeline must commit to a representation, and this choice (serialization, data schema, image, a specialized table encoder, or a graph) conditions everything downstream. The survey organized the field into schema understanding, the mapping of natural-language intent onto a structural blueprint, and data understanding, the problem of reasoning faithfully over table content, treating representation within the data pillar and singling out graph representation as a solid but so-far small-scale and document-centric approach whose extension to large tables is a central opportunity. Across both, a consistent picture emerged: single-pass methods have given way to multi-step pipelines that decompose, retrieve, verify, and correct, and the binding constraints are increasingly architectural (how structure and evidence are delivered to the model) rather than a matter of raw model capacity.

Chapter 3 presented our contribution to the schema-understanding pillar, Reflect-SQL [1], which threads self-reflection through all three stages of a Text-to-SQL pipeline: it constructs a semantic knowledge base over the schema, retrieves relevant elements iteratively, and synthesizes queries under syntactic and multi-dimensional semantic validation, using an LLM-as-a-judge to score correctness without a gold reference. Grounding reflection in the checkable signals that the SQL setting affords (execution, syntax, and result–intent entailment) allows the system to reach 72.03% execution accuracy on the industry-scale BIRD benchmark, and demonstrates that reference-free, self-critical generation is a viable path for structured tasks.

Chapter 4 then set out the research agenda that follows. On the schema side, the priorities are to make the knowledge base continually evolving rather than static, to scale retrieval by addressing pre-computed structure rather than searching at query time, to extend faithful synthesis to cross-database joins, and to make reflection both reliable and inexpensive. On the data-understanding side, our next pillar, the target task is Table Question Answering and the lead direction is to represent large tabular

data at scale, concretely as a graph, so that evidence retrieval becomes addressing rather than search, with robustness on complex table structures and higher-order reasoning as the targets and a large-scale Table QA benchmark as the instrument to measure them.

The unifying thesis of this report is that progress in tabular data understanding depends less on ever-larger models than on the architecture that supplies them with the right structure and the right evidence, and that this is above all what will let the field cross from small, curated benchmarks to the large, obscure, obsolescent data of real enterprises. Reflect-SQL is a concrete step in that direction for schema understanding; carrying the same principle into data understanding, so that both pillars hold at industry scale, is the trajectory we intend to pursue.

Bibliography

[1] A. Jain and M. Shrivastava, "Reflect-SQL: A self-reflection based framework for text-to-sql," in *Trends and Applications in Knowledge Discovery and Data Mining (PAKDD)*, ser. Lecture Notes in Computer Science. Singapore: Springer Nature Singapore, 2027, pp. 74–86.

[2] X. Wu, A. Ritter, and W. Xu, "Tabular data understanding with LLMs: A survey of recent advances and challenges," *arXiv preprint arXiv:2508.00217*, 2025.

[3] Y. Sui, M. Zhou, M. Zhou, S. Han, and D. Zhang, "Table meets LLM: Can large language models understand structured table data? a benchmark and empirical study," in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM)*, 2024.

[4] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," *arXiv preprint arXiv:1709.00103*, 2017.

[5] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[6] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai *et al.*, "MAC-SQL: A multi-agent collaborative framework for text-to-SQL," in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*, 2024.

[7] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei *et al.*, "CHASE-SQL: Multi-path reasoning and preference optimized candidate selection in text-to-SQL," *arXiv preprint arXiv:2410.01943*, 2024.

[8] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li *et al.*, "Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-SQLs (BIRD)," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[9] Z. Cheng, H. Dong, Z. Wang, R. Jia, J. Guo, Y. Gao, S. Han, J.-G. Lou, and D. Zhang, "HiTab: A hierarchical table dataset for question answering and natural language generation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

[10] Y. Zhao, Y. Li, C. Li, and R. Zhang, "MULTIHIERTT: Numerical reasoning over multi hierarchical tabular and textual data," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

[11] E. Lobo *et al.*, "Matching business glossaries to databases: A semantic and few-shot learning approach," in *Proceedings of the 2023 International Conference on Management of Data (SIGMOD)*, 2023.

[12] Y. Gao *et al.*, "Automatic metadata generation for text-to-SQL," *Proceedings of the VLDB Endowment*, 2025.

[13] Z. Ma *et al.*, "Enhancing text-to-SQL with domain knowledge injection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[14] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[15] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[16] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," *arXiv preprint arXiv:2310.11511*, 2023.

[17] X. Xu, C. Liu, and D. Song, "SQLNet: Generating structured queries from natural language without reinforcement learning," *arXiv preprint arXiv:1711.04436*, 2017.

[18] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

[19] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-SQL empowered by large language models: A benchmark evaluation (DAIL-SQL)," *Proceedings of the VLDB Endowment*, vol. 17, no. 11, 2024.

[20] R. Sun, S. O. Arik, A. Muzio, L. Miculicich, S. Gundabathula *et al.*, "SQLPrompt: In-context text-to-SQL with minimal labeled data," in *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

[21] Z. Ji, T. Yu, Y. Xu, N. Lee, E. Ishii, and P. Fung, "Towards mitigating LLM hallucination via self-reflection," in *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

[22] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[23] A. Su, A. Wang, C. Ye, C. Zhou, G. Zhang *et al.*, "TableGPT2: A large multimodal model with tabular data integration," *arXiv preprint arXiv:2411.02059*, 2024.

[24] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[25] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi, "G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[26] Q. Li, C. Huang, S. Li, Y. Xiang, D. Xiong, and W. Lei, "GraphOTTER: Evolving LLM-based graph reasoning for complex table question answering," in *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, 2025.

[27] P. Pasupat and P. Liang, "Compositional semantic parsing on semi-structured tables," in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2015.

[28] M. Iyyer, W.-t. Yih, and M.-W. Chang, "Search-based neural structured learning for sequential question answering," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017.

[29] F. Zhu, W. Lei, Y. Huang, C. Wang, S. Zhang *et al.*, "TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

[30] Z. Chen, W. Chen, C. Smiley, S. Shah, I. Borova *et al.*, "FinQA: A dataset of numerical reasoning over financial data," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[31] W. Chen, H. Zha, Z. Chen, W. Xiong, H. Wang, and W. Y. Wang, "HybridQA: A dataset of multi-hop question answering over tabular and textual data," in *Findings of the Association for Computational Linguistics: EMNLP*, 2020.

[32] V. Pal, A. Yates, E. Kanoulas, and M. de Rijke, "MultiTabQA: Generating tabular answers for multi-table question answering," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[33] L. Nan, C. Hsieh, Z. Mao, X. V. Lin, N. Verma *et al.*, "FeTaQA: Free-form table question answering," in *Transactions of the Association for Computational Linguistics*, 2022.

[34] W. Chen, H. Wang, J. Chen, Y. Zhang, H. Wang *et al.*, "TabFact: A large-scale dataset for table-based fact verification," in *International Conference on Learning Representations (ICLR)*, 2020.

[35] A. Parikh, X. Wang, S. Gehrmann, M. Faruqui, B. Dhingra *et al.*, "ToTTo: A controlled table-to-text generation dataset," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[36] Y. Zhao, Z. Qi, L. Nan, B. Mi, Y. Liu *et al.*, "QTSUMM: Query-focused summarization over tabular data," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[37] F. Lei, J. Chen, Y. Ye, R. Cao, D. Shin *et al.*, "Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows," in *arXiv preprint arXiv:2411.07763*, 2024.

[38] X. He, M. Zhou, X. Xu, X. Ma, R. Ding *et al.*, "Text2Analysis: A benchmark of table question answering with advanced data analysis and unclear queries," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[39] Y. Ye, B. Hui, M. Yang, B. Li, F. Huang, and Y. Li, "Large language models are versatile decomposers: Decomposing evidence and questions for table-based reasoning," in *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2023.

[40] Z. Wang, H. Zhang, C.-L. Li, J. M. Eisenschlos, V. Perot, Z. Wang, L. Miculicich, and T. Pfister, "Chain-of-table: Evolving tables in the reasoning chain for table understanding," in *International Conference on Learning Representations (ICLR)*, 2024.

[41] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[42] J. Herzig, P. K. Nowak, T. Müller, F. Piccinno, and J. Eisenschlos, "TAPAS: Weakly supervised table parsing via pre-training," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

[43] P. Yin, G. Neubig, W.-t. Yih, and S. Riedel, "TaBERT: Pretraining for joint understanding of textual and tabular data," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.